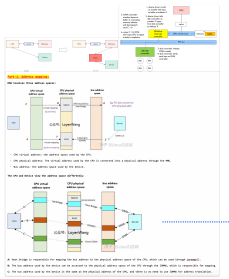
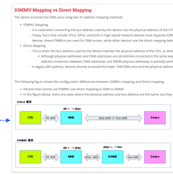
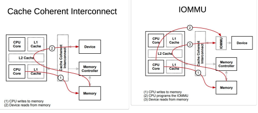
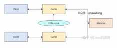





Cache Coherent Interconnect
IOMMU

Part-2: Cache Coherence:

DMA operations are usually related to cache. Let's first understand cache coherence:


Client
Cache
Coherency
Memory
Client
Cache
公众号: LoyenWang

Cache coherence device: The read and write between devices do not need to worry about cache consistency issues. The hardware will ensure data consistency. For example, the device connected to the ARM CCI port is a cache coherence device.

Non-coherence devices: require additional software operations (flush/invalidate) to ensure data consistency.

Note:

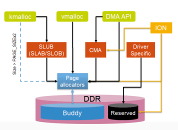
How do devices access the CPU's memory?

When a device accesses the CPU's memory, things become more complicated, and it is necessary to consider issues such as the synchronization between the device and the CPU, and between devices.

There are generally several ways for devices to access CPU memory, which are basically covered by the Linux kernel's DMA API:

- Direct access (DMA direct): The address that the device can access coincides with the address space of the CPU.
  When a large amount of continuous memory is required, there are two solutions:
    - static reserved memory: When the system starts, a piece of memory is reserved for the device
    - cma: A piece of memory is also allocated for device use, which can be global or device-specific. However, unlike the first solution, the system can use this memory when the device does not use it.
- If the addresses do not overlap, you can use iommu to make a mapping
- If there is no iommu, software is needed to do the mapping. For example, through swiotlb, a mapping will be made between the addresses that the device cannot access and the addresses that the device can access. This buffer is called a bounce buffer.

Kernel memory allocation


kmalloc
vmalloc
DMA API
ION
SLUB (SLAB/SLOB)
CMA
Driver Specific
Size > PAGE_SIZEx2
Page allocators
DDR
Buddy
Reserved

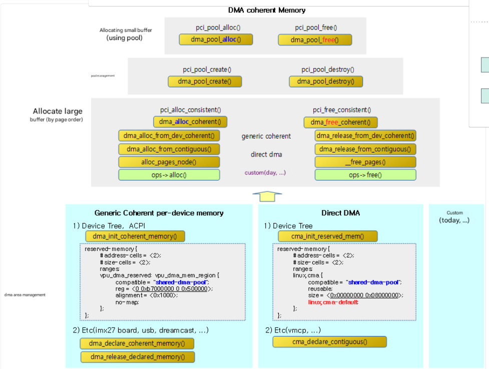
- kmalloc – for small chunks allocation, go through SLxB. Directly call alloc_pages for large continue size
- vmalloc – for virtual continuous memory allocation, physical is not required to be continuous (place in virtual vmalloc range)
- CMA – described in next page
- Reserved memory (in dts, reserved by memblock_alloc on boot) managed by drivers for specific usage (like GPU)
- ION – Android specific allocator, used for video/camera buffers

Buffer Types Based on Cache Coherency

- Coherent/Consistent DMA mapping
  - — Usually long lasting.
  - — Can be accessed by both ends.
  - — No-caching or uses cache coherent interconnect hardware*
  - — At least page sized
- Streaming DMA mapping
  - — Usually singly used and freed.
  - — Architecture/Platform optimized.
  - — Direction must be defined explicitly.

Coherent DMA Mappings

- dma_addr_t dma_handle;
- cpu_addr = dma_alloc_coherent(dev, size, &dma_handle, gfp);
  - returns two values:
    - the virtual address which you can use to access it from the CPU
    - dma_handle which you pass to thecard.
- Acts similar to __get_free_pages(), use pool option if memory size smaller than a page is needed
- dma_free_coherent(dev, size, cpu_addr, dma_handle);

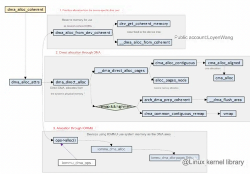

DMA coherent Memory
Allocating small buffer (using pool)
pci_pool_alloc()
dma_pool_alloc()
pci_pool_free()
dma_pool_free()
pool management
pci_pool_create()
dma_pool_create()
pci_pool_destroy()
dma_pool_destroy()
Allocate large buffer (by page order)
pci_alloc_consistent()
dma_alloc_coherent()
pci_free_consistent()
dma_free_coherent()
dma_alloc_from_dev_coherent()
dma_release_from_dev_coherent()
dma_alloc_from_contiguous()
dma_release_from_contiguous()
alloc_pages_node()
__free_pages()
ops->alloc()
ops->free()
generic coherent
direct dma
custom(dsp, ...)
Generic Coherent per-device memory
Direct DMA
1) Device Tree, ACPI
dma_init_coherent_memory()
cma_init_reserved_mem()
"shared-dma-pool"
linux,cma-default
2) Etc(mx27 board, usb, dreamcast, ...)
dma_declare_coherent_memory()
dma_release_declared_memory()
2) Etc(vmcp, ...)
cma_declare_contiguous()
Custom (today, ...)
dma area management

Part-3: DMA mappings:

The Linux kernel provides two DMA mapping interfaces: Consistent mapping and Stream mapping.

3.1 Consistent DMA mappings:

- Consistent mapping: corresponds to cache-coherence devices, where the hardware ensures that both the device and the CPU can access data in parallel and see each other's updates without the need for software flush operations.
- Usually the map operation is performed when the driver Init, and the un-map operation is performed when de-init.
- Typically when using consistent DMA mapping, you first need to allocate an area through the "dma_alloc_coherent" interface:

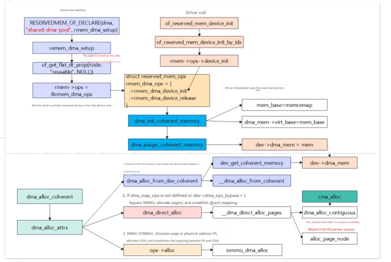

dma_alloc_coherent
1. Prioritize allocation from the device-specific dma pool
dma_alloc_from_dev_coherent
dev_get_coherent_memory
__dma_alloc_from_coherent
Public account:LoyenWang
2. Direct allocation through DMA
dma_alloc_attrs
dma_direct_alloc
__dma_direct_alloc_pages
dma_alloc_contiguous
cma_alloc_aligned
cma_alloc
alloc_pages_node
arch_dma_prep_coherent
__dma_flush_area
remap && highmem
dma_common_contiguous_remap
vmap
3. Allocation through IOMMU
ops->alloc()
iommu_dma_alloc
iommu_dma_ops
@Linux kernel library

dma_alloc_coherent is used to allocate coherent memory and return the corresponding virtual address.

There are three ways to allocate memory:

1) Allocate from the device-specific DMA pool first.
2) If there is no dedicated DMA pool, if it is DMA-direct access, it is allocated through "dma_direct_alloc", and the underlying layer relies on CMA for allocation.
3) For devices using IOMMU, it is allocated through the IOMMU operation function set.

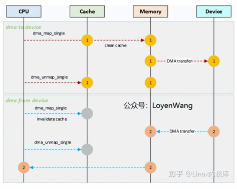
function	illustrate
void *dma_alloc_coherent(struct device *dev, size_t size, dma_addr_t *dma_handle, gfp_t gfp)	This function will allocate a piece of memory, return the virtual address, and the physical address The dma handle is returned to the caller. dev: requested device; size: requested memory size; dma_handle: returns address 10, the address accessed by the driver hardware; gfp: requested flag, whether to block, etc.
void dma_free_coherent(struct device *dev, size_t size, void *cpu_addr, dma_addr_t dma_handle)	Releases the allocated memory. cpu_addr is the virtual address returned by the allocation.


Device tree matching
RESERVEDMEM_OF_DECLARE(dma, "shared-dma-pool", rmem_dma_setup)
remem_dma_setup
of_get_flat_dt_prop(node, "reusable", NULL))
rmem->ops = &rmem_dma_ops
Driver call
of_reserved_mem_device_init
of_reserved_mem_device_init_by_idx
rmem->ops->device_init
struct reserved_mem_ops rmem_dma_ops = { .=rmem_dma_device_init .=rmem_dma_device_release }
dma_init_coherent_memory
mem_base=memremap
dma_mem->virt_base=mem_base
dma_assign_coherent_memory
dev->dma_mem = mem
dev_get_coherent_memory
dev->dma_mem
dma_alloc_coherent
dma_alloc_from_dev_coherent
__dma_alloc_from_coherent
dma_alloc_attrs
dma_direct_alloc
__dma_direct_alloc_pages
cma_alloc
dma_alloc_contiguous
alloc_page_node
ops->alloc
iommu_dma_alloc

3.2 Streaming DMA mappings:

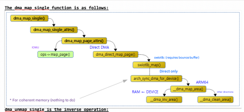
Streaming mapping: corresponds to non-coherence devices.

Let's first look at the data consistency issue:

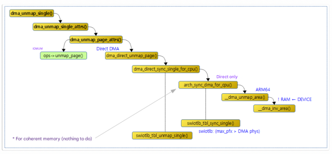

CPU
Cache
Memory
Device
dma to device
dma from device
公众号: LoyenWang

- When DMA to device, the data in the cache needs to be flushed to the memory.
- When DMA from device, the data in the cache needs to be invalidated to prevent the CPU from reading the old data.

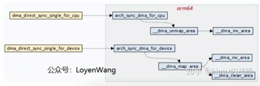
The dma_map_single is as follows:



The dma_unmap_single is the inverse operation:


* For coherent memory (nothing to do)
swiotlb_tbl_unmap_single()
swiotlb_tbl_sync_single()
swiotlb: (max_pfx > DMA phys)

As can be seen from the above functions, arch-related cache operations will eventually be called, which is strongly related to the architecture. Take arm64 as an example:


arm64
dma_direct_sync_single_for_cpu
arch_sync_dma_for_cpu
__dma_unmap_area
__dma_inv_area
dma_direct_sync_single_for_device
arch_sync_dma_for_device
__dma_map_area
__dma_inv_area
__dma_clean_area
公众号: LoyenWang

Streaming DMA Mappings

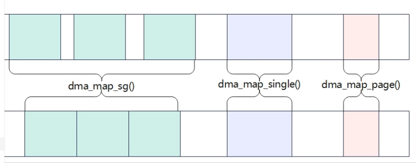
- Map Single Region

```
struct device *dev = &my_dev->dev;
dma_addr_t dma_handle;
void *addr = buffer->ptr;
size_t size = buffer->len;

dma_handle = dma_map_single(dev, addr, size, direction);
if (dma_mapping_error(dev, dma_handle)) {
/* reduce current DMA mapping usage, delay and try again later or reset driver. */
    goto map_error_handling;
}
```

- Unmap Single Region

```
dma_unmap_single(dev, dma_handle, size, direction);
```


dma_map_sg()
dma_map_single()
dma_map_page()

Streaming DMA Mappings

- Map Scatterlist

```
int i, count = dma_map_sg(dev, sglist, nents, direction);
struct scatterlist *sg;

for_each_sg(sglist, sg, count, i) {
    hw_address[i] = sg_dma_address(sg);
    hw_len[i] = sg_dma_len(sg);
}
```

- Unmap Scatterlist

```
dma_unmap_sg(dev, sglist, nents, direction);
```

DMA directions

You can specify the direction in which data moves during DMA transfer as follows.

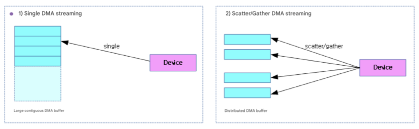
- DMA_BIDIRECTIONAL
  - Main memory and device bidirectional
  - Specifies that bidirectional transmission is possible when the exact direction is unknown. However, the cost is higher than that of unidirectional transmission.
- DMA_TO_DEVICE
  - Device orientation in main memory
- DMA_FROM_DEVICE
  - Main memory direction on the device
- DMA_NONE
  - Use when debugging

single vs sg(scatter/gather)

Streaming mapping performs mapping and unmapping on a one-shot basis for each DMA request, and supports two versions:

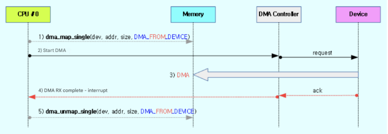
- single map/unmap
  - Used when performing DMA transfer targeting a physically contiguous area.
    - Example) Ring buffer consisting of large areas
  - Limitation: highmem area cannot be used.
  - Call dma_map_single() before starting a DMA transfer, and call dma_unmap_single() when the DMA transfer is complete.
- scatter/gather map/unmap
  - Used when performing DMA transfers simultaneously to multiple fragmented buffers that are not physically contiguous.
    - It has the advantage of not having to prepare a large, physically contiguous DMA buffer.
  - The individual memory areas specified in the scatterlist must use physically contiguous memory, but the entire DMA buffer is not contiguous and is fragmented.
  - Requests the offset and length information of the area to be used as a DMA buffer in the scatterlist array.
  - Call dma_map_sg() before starting a DMA transfer, and call dma_unmap_sg() when the DMA transfer is complete.

The following figures show single DMA streaming and scatter/gather DMA streaming.


1) Single DMA streaming
single
Device
Large contiguous DMA buffer
2) Scatter/Gather DMA streaming
scatter/gather
Device
Distributed DMA buffer

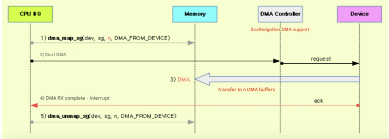
The following figure shows streaming DMA mapping from the device to memory over a single contiguous memory region.

Single DMA Streaming - (ex: Read Disk)


CPU #0
Memory
DMA Controller
Device
1) dma_map_single(dev, addr, size, DMA_FROM_DEVICE)
2) Start DMA
request
3) DMA
4) DMA RX complete - interrupt
ack
5) dma_unmap_single(dev, addr, size, DMA_FROM_DEVICE)

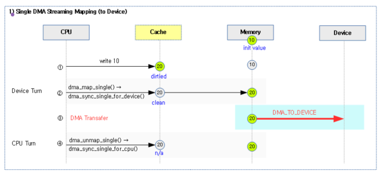
The following figure shows streaming DMA mapping from the device to memory across multiple fragmented memory regions.

Scatter/gather DMA streaming (ex: Read Disk)

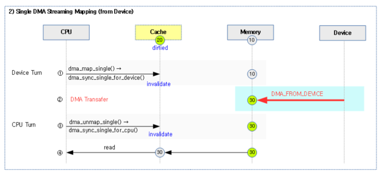

CPU #0
Memory
DMA Controller
Device
Scatter/gather DMA support
1) dma_map_sg(dev, sg, n, DMA_FROM_DEVICE)
2) Start DMA
request
3) DMA
Transfer to n DMA buffers
4) DMA RX complete - interrupt
ack
5) dma_unmap_sg(dev, sg, n, DMA_FROM_DEVICE)

streaming dma – single map/unmap

The following figure shows a single streaming mapping/unmapping process for cache when DMA transferring from RAM to Device.


1) Single DMA Streaming Mapping (to Device)
CPU
Cache
Memory
Device
10
init value
①
write 10
20
dirtied
Device Turn
②
dma_map_single() →
dma_sync_single_for_device()
clean
③
DMA Transafer
DMA_TO_DEVICE
CPU Turn
④
dma_unmap_single() →
dma_sync_single_for_cpu()
n/a

The following figure shows a single streaming mapping/unmapping process for a cache when DMA transferring from the device to RAM.


2) Single DMA Streaming Mapping (from Device)
CPU
Cache
Memory
Device
20
dirtied
10
Device Turn
①
dma_map_single() →
dma_sync_single_for_device()
invalidate
②
DMA Transafer
DMA_FROM_DEVICE
30
CPU Turn
③
dma_unmap_single() →
dma_sync_single_for_cpu()
invalidate
④
read

DMA -4- (DMA Mapping)

If you use Streaming mapping instead of Coherent mapping, you will need to map and unmap (sync operation) every time before and after DMA transfer.

- Performs cache sync operations and iommu mapping/unmapping before/after DMA.
  - Cleans or invalidates the cache depending on the DMA direction.
  - Devices using DMA perform mapping and unmapping when using iommu.
- A bounce buffer is required when the DMA area is smaller than the system DRAM.
  - In these cases, we use swiotlb, which requires data copy.
  - The default bounce buffer is 64MB.
    - A bounce buffer is allocated within the accessible DMA address range.
    - For ARM64, if you use ZONE_DMA32, you are limited to 4GB physical addresses.

DMA Mapping Type

Select and use one of the two types of DMA mapping as follows:

- constant dma mapping
  - No individual mapping is required during driver initialization, so no mapping is performed before/after DMA transmission.
- streaming dma mapping
  - Use when mapping is required before/after DMA transmission.

Streaming DMA mapping types

- Single
  - Uses a single physically contiguous DMA buffer.
- Scatter/Gather
  - Uses physically distributed DMA buffers.
  - Allows transfer to multiple DMA buffers in a single DMA operation, operating as if they were physically contiguous DMA buffers.
  - This feature is only available on advanced DMA controllers.

DMA Mapping

Use one of two mapping strategies.

- consistent dma mapping
  - If dma mapping (iommu and cache sync) is required during device driver initialization, pre-mapping is performed.
  - No need to perform individual dma mappings until driver termination
- streaming dma mapping
  - When using iommu, mapping/unmapping is involved.
  - If coherent memory is not used, a cache sync function is provided.

streaming dma mapping

The following methods are provided to utilize DMA buffers:

- Single Mapping
  - It is a request for one area,
- Page Mapping
  - This is a request for one page.
- scatter/gather mapping
  - This is a request for multiple fragmented areas.

for_cpu

If sync is required before DMA

- You can use the dma_sync_*_for_cpu() functions,

for_device

If sync is required after DMA

- You can use the dma_sync_*_for_device() functions.

- dma_map_single_attrs()
- dma_unmap_single_attrs()
- dma_map_single()
- dma_unmap_single()
- dma_map_page()
- dma_unmap_page()
- dma_map_sg_attrs()
- dma_unmap_sg_attrs()
- dma_map_sg()
- dma_unmap_sg()
- dma_mapping_error()
- dma_sync_single_for_cpu()
- dma_sync_single_for_device()
- dma_sync_single_range_for_cpu()
- dma_sync_single_range_for_device()
- dma_sync_sg_for_cpu()
- dma_sync_sg_for_device()

IOMMU

This is an API that supports mapping after allocating an iommu domain and attaching a device that uses it.

- iommu_domain_alloc()
- iommu_domain_free()
- iommu_attach_device()
- iommu_detach_device()
- iommu_map()
- iommu_unmap()

Part-4 : DMA Address Limitations:

DMA Address Limitations

Even if the physical address or bus address has a range that can be expressed as a 64-bit address, there are cases where the DMA addressing of the device is limited, as in the following examples.

- When using 32bit PCI devices on a 64bit PCIX bus
- If the device is connected via the ISA bus on a legacy x86 system.
  - DMA access is possible using only 20-bit (1M) or 24-bit (16M) addresses.

The following three APIs are typically used when setting DMA address restrictions.

- dma_set_mask()
  - Used when using streaming mapping.
  - Example) If 32bit (4G) limitation is required
    - mask=0xffff_ffff
    - or DMA_BIT_MASK(32)
- dma_set_coherent_mask()
  - Used in consistent mapping.
  - The coherent mask can use a mask value that is the same as the limit used in dma_set_mask() or a smaller mask value.
- dma_set_mask_and_coherent()
  - This is a configuration that uses the above two with the same mask value.

To understand DMA (Direct Memory Access) and how the addressable range changes with different bit widths, let's break it down into concepts, followed by examples with block diagrams.

Key Concepts

1. **DMA Addressing Range**:
   - A device's DMA capability depends on the number of address bits it supports.
   - For a device with an `N`-bit DMA mask, it can access a physical memory range of `2^N` bytes.
     - For 24-bit: $2^{24} = 16\,\text{MB}$
     - For 32-bit: $2^{32} = 4\,\text{GB}$
2. **DMA Translation**:
   - Modern devices use **IOMMUs** or **address translation units** to map limited device-accessible memory into larger CPU physical memory ranges.

Examples

Example 1: 24-bit DMA

- Device DMA mask: 24-bit (`DMA_BIT_MASK(24)`).
- Addressable memory range: $0x000000 \text{ to } 0xFFFFFF$ (16 MB).
- If the system has physical memory beyond 16 MB, only the first 16 MB can be accessed by the device without IOMMU support.

```plaintext
Physical Memory (32 MB total):

+------------------------+----------------------+
|   Address Range        | Description          |
+------------------------+----------------------+
| 0x00000000 - 0x00FFFFFF| Accessible by device |
| 0x01000000 - 0x01FFFFFF| NOT accessible       |
+------------------------+----------------------+
```

Example 2: 32-bit DMA

- Device DMA mask: 32-bit (`DMA_BIT_MASK(32)`).
- Addressable memory range: $0x00000000 \text{ to } 0xFFFFFFFF$ (4 GB).
- If the system has more than 4 GB physical memory, the device cannot access addresses beyond 4 GB unless IOMMU is configured.

```plaintext
Physical Memory (8 GB total):

+------------------------+----------------------+
|   Address Range        | Description          |
+------------------------+----------------------+
| 0x00000000 - 0xFFFFFFFF| Accessible by device |
| 0x100000000 - Beyond   | NOT accessible       |
+------------------------+----------------------+
```

Block Diagram Representation

Here's a block diagram illustrating the difference between 24-bit and 32-bit DMA devices accessing memory:

```plaintext
+--------------------------------+        +--------------------------------+
|         CPU Memory             |        |         CPU Memory             |
| 32 MB Total                    |        | 8 GB Total                     |
|                                |        |                                |
| +----------------------------+ |        | +----------------------------+ |
| | 0x00000000 - 0x00FFFFFF    | | <--+   | | 0x00000000 - 0xFFFFFFFF    | | <--+
| |  (Accessible by Device)    | |    |   | |  (Accessible by Device)    | |    |
| +----------------------------+ |    |   | +----------------------------+ |    |
| | 0x01000000 - Beyond        | |    |   | | 0x100000000 - Beyond       | |    |
| |  (NOT Accessible)          | |    |   | |  (NOT Accessible)          | |    |
| +----------------------------+ |    |   | +----------------------------+ |    |
+--------------------------------+    |   +--------------------------------+    |
                                      |                                    |
+--------------------------------+    |   +--------------------------------+    |
|  Device with 24-bit DMA Mask   | ---+   |  Device with 32-bit DMA Mask  | ---+
| (Max range: 16 MB)             |        | (Max range: 4 GB)             |
+--------------------------------+        +--------------------------------+
```

Why Does This Matter?

1. **Limitation without IOMMU**:
   - Devices with smaller DMA masks (e.g., 24-bit) cannot directly access all system memory in high-memory configurations.
2. **Solution**:
   - Systems with **IOMMU** can remap higher memory ranges into the device-accessible range.
   - Example:
     - Device wants to access a buffer at $0x01000000$.
     - IOMMU remaps it to a device-usable range within $0x00000000 - 0x00FFFFFF$.

References:

1. https://www.laumy.tech/919.html
2. https://www.cse.iitd.ac.in/~rijurekha/col788/dma.pdf
3. https://ctf.re/kernel/pcie/tutorial/dma/mmio/tlp/2024/03/26/pcie-part-2/
4. https://iosoft.blog/2020/05/25/raspberry-pi-dma-programming/
5. https://joyxu.github.io/2021/05/25/linux-device-memory/